

LAB DAY 1

Hands-On Afternoon Labs

Two labs, roughly two hours each, building directly on this morning's material

Lab 1: Data Summarizer

Lab 2: Parallel Batch Processor

Pairs Welcome

Docs Yes / LLMs No

Everything in both labs draws on Day 1's lecture modules — this is where that material becomes muscle memory.

Two labs, one structure

- **Each lab opens with a short framing and requirements briefing,** then roughly 90 minutes of independent or paired work, then a group walkthrough of reference solutions.
- **Reference solutions show more than one valid approach on purpose —** there's rarely one right way to write a piece of Rust, and comparing approaches is where a lot of the real learning happens.
- **Both labs are scoped to run about two to two-and-a-half hours end to end, including discussion and presentations.**

Pairs welcome. Docs encouraged. LLMs, please don't.

Work in pairs or small groups if you'd like — talking through a borrow-checker error out loud is often the fastest way to understand it. Please don't ask an LLM to write or fix your code today: the goal is to build your own intuition for ownership, errors, and concurrency, and that's exactly what an LLM shortcut skips past. Internet search is fully fair game — the standard library docs, docs.rs, the Rust Book, and users.rust-lang.org are exactly what a working Rust engineer reaches for. If you're stuck for more than about ten minutes, flag an instructor — that's part of the lab, not a failure of it.

TODAY'S SCHEDULE

Where labs fit in a full course day

Block	Duration	Content
Morning lecture	~3.5–4 hrs (with breaks)	Modules 1–16 from Day 1: Foundations
Lunch	~45–60 min	—
Lab 1	~2.5 hrs	Robust Data Summarizer — ownership, error handling, iterators
Short break	~10–15 min	—
Lab 2	~2.5 hrs	Parallel Batch Processor — concurrency, borrowing
Total	~7.5–8.5 hrs	A full course day

01

Lab 1: Robust Data Summarizer

Ownership, error handling, and iterators — applied to messy real-world input

Lab 1 time budget

Segment	Duration	What happens
Setup & framing	10 min	Instructor presents the problem, requirements, and ground rules; questions answered
Independent / paired work	90 min	Students work solo or in pairs; instructors circulate to help, not to hand out answers
Short break	10 min	Stretch, refill coffee
Solution walkthrough & discussion	20 min	Instructor presents reference approaches; group discussion questions
Student presentations	20–30 min	2–3 pairs briefly present their approach and one thing they learned

The scenario

- **Every run, an instrument logs a batch of readings to a plain text file —** one record per line, as "id,value".
- **Some lines are malformed — a dropped byte, a corrupted field, a stray blank line —** because the hardware isn't perfect and the log format was never validated at write time.
- **You need a small command-line tool that summarizes the good data and doesn't fall over on the bad.**

LAB 1

Sample input file

```
S1,23.4  
S2,25.1  
S3,BAD  
S4,22.9  
  
S5,24.0
```

Note: Line 3 has a non-numeric value; line 5 is blank. Both are realistic hardware-log failure modes, and both must be handled without panicking.

LAB 1

Expected output (illustrative)

```
Records read:      6
Valid records:     4
Skipped (bad):     2
Mean:             23.85
Min:              22.90
Max:              25.10
Std dev:          0.90
```

Note: Exact formatting is your choice — these numbers are illustrative, not a byte-for-byte spec to match.

Requirements

In rough priority order — get the top ones solid before chasing the rest.

- **Read the file path from the first command-line argument, parse each line as "id,value".**
- **Skip and count malformed lines (empty, missing comma, non-numeric value) without panicking —** print a one-line warning to stderr for each one skipped.
- **Compute and print: total lines read, valid count, skipped count, mean, min, max, and standard deviation.**
- **Group records by the first character of their id and print a per-group mean.**
- **Include at least two unit tests: one for the stats calculation on a known small dataset, one for malformed-line handling.**

What we'll look for

Criterion	What we're checking
Correctness	Stats match a hand-computed reference on the sample input above
Robustness	No panics on any malformed line — the program always finishes and reports a result
Idiomatic style	cargo clippy runs clean; ownership and error handling follow today's patterns, not C++ habits ported over
Tests	At least two tests, both passing, covering a normal case and a malformed-input case

Stretch goals

Optional, for pairs who finish early — pick at most one or two.

- Accept input from stdin when no file argument is given (or when the argument is "-").
- Print the top three outlier records — the ones furthest from the mean.
- Replace manual argument parsing with the clap crate — look up its docs.rs page.
- Define a custom error enum implementing `std::error::Error` instead of silently dropping bad lines.
- Generate a synthetic 1-million-line file and report how long your program takes to summarize it.

Hints, if you want them

Socratic on purpose — look these up rather than guess.

- **`str::split_once`, `str::trim`, and `"3.14".parse::<f64>()` do most of the parsing work for you.** What type does `parse()` return, and what does `.ok()` do to it?
- **`Iterator::fold` can compute a running (count, sum, sum_sq) in one pass instead of three separate passes.**
- **`HashMap::entry(...).or_insert_with(...)` is the idiomatic way to build a `HashMap<String, Vec<f64>>` without a manual `contains_key` check.**
- **Searching "rust parse csv line without a crate" or the `std::str` documentation this afternoon is completely fair game.**

Try it yourself now — reference solutions follow.

This is a good place to pause the slides for the 90-minute work session. What follows are two full reference approaches, plus a third for early finishers — not the only correct answers, just a solid baseline to compare against.

Discussion questions

For the group discussion block — no single right answer expected.

● **Which approach would you extend to stream a 10 GB file without loading it all into memory first?** What single method call would you swap in to get there?

● **Where did your program panic before you added error handling —** and why didn't the compiler warn you about that possibility?

● **If tomorrow's task were "parallelize this," which approach's internal data structures would need to change the least?**

Presentation prompts

Three quick things for each presenting pair to cover — five minutes, tops.

● Which approach did you take, and why?

● Show us one bug you hit and how you found it.

● What's one thing you'd do differently with another hour?

02

Lab 2: Parallel Batch Processor

Concurrency and borrowing — proving a parallel version matches a trusted sequential one

LAB 2: PARALLEL BATCH PROCESSOR

Lab 2 time budget

Segment	Duration	What happens
Setup & framing	10 min	Instructor presents the problem, requirements, and ground rules; questions answered
Independent / paired work	90 min	Students work solo or in pairs; instructors circulate to help, not to hand out answers
Short break	10 min	Stretch, refill coffee
Solution walkthrough & discussion	20 min	Instructor presents reference approaches; group discussion questions
Student presentations	20–30 min	2–3 pairs briefly present their approach and one thing they learned

The scenario

- Your team occasionally needs to validate a large batch buffer — think a big block of raw bytes off an acquisition device — before it's trusted downstream.
- A single-threaded pass works and is already trusted, but it's too slow on the largest buffers your team sees.
- You've been asked for a parallel version that produces byte-for-byte identical results to the sequential one, just faster — not "probably the same," identical, provably.

LAB 2

The trusted reference implementation (given)

```
struct BatchReport { len: usize, sum: u64, min: u8, max: u8, xor_checksum: u8 }

fn analyze_sequential(data: &[u8]) -> BatchReport {
    let mut sum = 0u64;
    let mut min = u8::MAX;
    let mut max = 0u8;
    let mut xor_checksum = 0u8;
    for &b in data {
        sum += b as u64;
        min = min.min(b);
        max = max.max(b);
        xor_checksum ^= b;
    }
    BatchReport { len: data.len(), sum, min, max, xor_checksum }
}
```

Your task: This function is your specification and your correctness oracle. Your job is `analyze_parallel(data, num_workers)` — same return type, same results, using threads.

Requirements

- **Split data into `num_workers` roughly-equal contiguous chunks** — handle `data.len()` not dividing evenly, and `num_workers` larger than `data.len()`.
- **Use `std::thread::scope` so threads can borrow slices of data directly** — no cloning the buffer, no `Arc`, no `unsafe`.
- **Each thread computes a partial report for its chunk; combine partial reports into one final `BatchReport`.**
- **The result must exactly match `analyze_sequential(data)` for the same data, for every `num_workers` you test.**
- **Include a test that checks this equality across at least three different worker counts, including one that doesn't evenly divide the data.**

What we'll look for

Criterion	What we're checking
Correctness	Output matches <code>analyze_sequential</code> exactly, for every worker count tested
Safe concurrency	No unsafe blocks; no <code>Arc<Mutex<_>></code> where scoped borrowing or a channel would do
Edge cases	Empty data, <code>num_workers = 1</code> , and <code>num_workers > data.len()</code> all handled without panicking
Tests	The equivalence test passes across multiple worker counts, not just one

Stretch goals

- Report live progress as chunks complete, using a channel or a shared atomic counter.
- Measure and print wall-clock time for both the sequential and parallel versions on the same large input — at what size does parallel actually win?
- Pick `num_workers` automatically using `std::thread::available_parallelism()`.

Hints, if you want them

- **`std::thread::scope` lets spawned threads borrow data with an ordinary `&` reference** — look up why that's sound here when a plain `thread::spawn` would require `'static`.
- **`slice::chunks(n)` already handles an uneven final chunk correctly** — the harder part is computing the right chunk size in the first place. Does `usize` have a method for "divide, rounding up"?
- **A `thread::scope` closure's `JoinHandle::join()` returns whatever the spawned closure returns, directly and typed** — no shared mutable state required just to get a value back out.

Try it yourself now — reference solutions follow.

Same structure as Lab 1: two full approaches with different trade-offs, a correctness test you can compare against your own, and a set of common pitfalls to check your solution against.

Discussion questions

- **This program parallelizes buffer processing without a single unsafe block, and a data race isn't possible even if you tried to introduce one by accident.** What would the equivalent C++ version need to get right by hand that Rust enforced here for free?
- **sum, min, max, and xor_checksum are all associative and commutative — partial results combine in any order.** What would change in your design if the spec required an order-dependent rolling hash instead?
- **If you measured it: at what data size did the parallel version actually beat the sequential one?** Why might a small input make threading slower, not faster?

Presentation prompts

● Which approach did you take, and why?

● Show us one bug you hit and how you found it.

● What's one thing you'd do differently with another hour?